

SATELLITE FLIGHT COMPUTER

STEP-BY-STEP PROCESS GUIDE

Document 4 of 4 — Every action in exact order. No code. No guessing. Just steps.

HOW TO USE THESE FOUR DOCUMENTS TOGETHER

Document 1 — Main Guide : Learn concepts and context for each phase
Document 2 — Missing Files : Extra detail on linker, startup, UART, and build system
Document 3 — Code Reference : Open this while coding. Every file, complete, ready to copy.
Document 4 — This document : Your daily driver. Follow each numbered step in order.
When a step says "create file X", look it up in Document 3.

You do NOT need to read Documents 1 and 2 to use Documents 3 and 4.
Documents 3 and 4 are fully self-contained for building the project.

Before You Start

This guide has **6 phases and 47 numbered steps**. Each step tells you exactly what to do. Green boxes tell you what to verify before moving on. If a verification fails, do not proceed — fix it first.

RULE 1: Never skip a verification box. They catch problems early before they become hard to debug.

RULE 2: When a step says "create file X — see Doc 3", open Document 3, find that file, and copy it exactly.

RULE 3: Every command shown is the exact command to run. Copy it as-is.

RULE 4: If something breaks, check the troubleshooting section in Document 2 first.

Full Timeline at a Glance

Phase	Title	Timeline	Hours	Deliverable
Phase 1	Environment Setup	Week 1	6-8 hrs	Install tools, verify toolchain, Hello World in QEMU
Phase 2	FreeRTOS Basics	Week 2	8-10 hrs	Configure RTOS, two tasks running, queue communication
Phase 3	Flight Computer Core	Weeks 3-4	10-14 hrs	All 4 tasks, ADCS at 50 Hz, telemetry packets over UART
Phase 4	Fault Detection	Week 5	8-10 hrs	Health monitor, watchdog, fault flags, safe mode
Phase 5	Ground Station GUI	Week 6	8-10 hrs	Python GUI with live plots, command uplink
Phase 6	Polish & Ship	Week 7	6-8 hrs	Tests, README, GitHub, demo recording

PHASE 1 — Environment Setup

Week 1
timeline

6-8 hrs
hrs/phase

Install every tool you need. Verify each one works before moving on. Do not skip any verification.

STEP 1 Install VS Code

Go to code.visualstudio.com and download VS Code for your OS. Run the installer.

✓ VERIFY BEFORE CONTINUING

- ☐ VS Code opens without error
- ☐ You can create a new file

STEP 2 Install the ARM cross-compiler

Linux (Ubuntu/Debian):

```
sudo apt update
sudo apt install -y gcc-arm-none-eabi gdb-multiarch binutils-arm-none-eabi
```

macOS:

```
brew install --cask gcc-arm-embedded
```

Windows: Download the installer from developer.arm.com/downloads/-/gnu-rm. Run it and tick "Add to PATH".

✓ VERIFY BEFORE CONTINUING

- ☐ Running: `arm-none-eabi-gcc --version` prints a version number (not an error)

STEP 3 Install QEMU

Linux:

```
sudo apt install -y qemu-system-arm
```

macOS:

```
brew install qemu
```

Windows: Download from qemu.org/download/#windows. Install and add the folder to your PATH.

✓ VERIFY BEFORE CONTINUING

- ☐ Running: `qemu-system-arm --version` prints a version number

STEP 4 Install CMake and Make

Linux:

```
sudo apt install -y cmake make ninja-build
```

macOS:

```
brew install cmake make ninja
```

Windows: Download CMake from cmake.org. During install, select "Add to system PATH".

✓ **VERIFY BEFORE CONTINUING**

- ☐ `cmake --version` prints a version number
- ☐ `make --version` prints a version number

STEP 5 Install Git

Linux: `sudo apt install -y git`

macOS: `brew install git`

Windows: Download from git-scm.com. Use default options during install.

✓ **VERIFY BEFORE CONTINUING**

- ☐ `git --version` prints a version number

STEP 6 Install Python and GUI dependencies

Linux:

```
sudo apt install -y python3 python3-pip python3-tk
pip3 install pyserial matplotlib
```

macOS:

```
brew install python python-tk
pip3 install pyserial matplotlib
```

Windows: Download from python.org. During install, tick "Add Python to PATH". Then:

```
pip install pyserial matplotlib
```

✓ **VERIFY BEFORE CONTINUING**

- ☐ `python3 --version` (or `python --version` on Windows) prints 3.10 or higher
- ☐ `python3 -c "import tkinter; tkinter.Tk()"` opens a blank window and closes without error

STEP 7 Install VS Code extensions

Open VS Code. Press **Ctrl+Shift+X** to open Extensions. Search and install each of these:

C/C++ by Microsoft

CMake Tools by Microsoft

Cortex-Debug by marus25

Serial Monitor by Microsoft

Python by Microsoft

GitLens by GitKraken

✓ **VERIFY BEFORE CONTINUING**

- ☐ All 6 extensions appear in the Extensions sidebar under "Installed"

STEP 8 Install socat (virtual serial port tool)

Linux:

```
sudo apt install -y socat
```

macOS:

```
brew install socat
```

Windows: socat is Linux-only. On Windows use [com0com](#) instead (free, from sourceforge). This tool is needed in Phase 5.

✓ **VERIFY BEFORE CONTINUING**

- socat -h prints help text (Linux/macOS)

STEP 9 Create the project folder structure

Open a terminal and run these commands exactly:

```
mkdir satellite-flight-computer
cd satellite-flight-computer
git init
mkdir -p src/tasks src/drivers src/common ground_station tests docs
```

✓ **VERIFY BEFORE CONTINUING**

- You are inside the satellite-flight-computer/ folder
- Running: ls shows: src/ ground_station/ tests/ docs/

STEP 10 Clone FreeRTOS kernel into the project

Run this from inside your **satellite-flight-computer/** folder:

```
git clone https://github.com/FreeRTOS/FreeRTOS-Kernel.git FreeRTOS
```

✓ **VERIFY BEFORE CONTINUING**

- A FreeRTOS/ folder now exists inside your project
- Running: ls FreeRTOS/Source/ shows: tasks.c queue.c list.c timers.c include/
- Running: ls FreeRTOS/Source/portable/GCC/ shows a folder called ARM_CM3

STEP 11 Write and run a Hello World sanity check

Before adding FreeRTOS, verify your toolchain and QEMU work with the simplest possible program.

Create a temporary file **test_hello.c** anywhere (not in your project yet) with this exact content:

```
#include <stdint.h>
#define UART0_DR (*(volatile uint32_t *)0x4000C000)
void uart_puts(const char *s) { while (*s) UART0_DR = (uint32_t)(*s++); }
void main(void) { uart_puts("Hello from ARM\n"); for(;;){} }
```

Compile it:

```
arm-none-eabi-gcc -mcpu=cortex-m3 -mthumb -nostdlib -e main test_hello.c -o
test_hello.elf
```

Run it in QEMU:

```
qemu-system-arm -M lm3s6965evb -kernel test_hello.elf -serial mon:stdio -nographic
```

✓ **VERIFY BEFORE CONTINUING**

- ❑ Terminal prints: Hello from ARM
- ❑ Press Ctrl+A then X to quit QEMU
- ❑ If this works, your toolchain and QEMU are correctly installed

PHASE 2 — FreeRTOS Basics

Week 2
timeline

8-10 hrs
hrs/phase

Create the project scaffold and get FreeRTOS running with two simple tasks. No flight computer logic yet — just prove the scheduler works.

STEP 12 Create `flight_types.h`

Open Document 3. Find `src/common/flight_types.h`. Create that file at that exact path inside your project. Copy the code exactly.

✓ VERIFY BEFORE CONTINUING

- ☐ File exists at `src/common/flight_types.h`
- ☐ No copy-paste errors (check the `#endif` at the bottom is there)

STEP 13 Create `FreeRTOSConfig.h`

Open Document 3. Find `src/FreeRTOSConfig.h`. Create that file. This is the RTOS kernel configuration.

Important: `FreeRTOSConfig.h` must be inside an include path that the FreeRTOS source files can see. The `CMakeLists.txt` (created in Step 18) adds `src/` as an include path, so placing it at `src/FreeRTOSConfig.h` is correct.

✓ VERIFY BEFORE CONTINUING

- ☐ File exists at `src/FreeRTOSConfig.h`

STEP 14 Create `linker.ld`

Open Document 3. Find `linker.ld`. Create this file at the **root of your project** (same level as the `src/` folder, NOT inside it).

✓ VERIFY BEFORE CONTINUING

- ☐ File exists at `satellite-flight-computer/linker.ld` (root level, not inside `src/`)

STEP 15 Create `startup.c`

Open Document 3. Find `src/startup.c`. Create this file. It defines the interrupt vector table and the reset handler that runs before `main()`.

✓ VERIFY BEFORE CONTINUING

- ☐ File exists at `src/startup.c`

STEP 16 Create the UART driver

Open Document 3. Create both UART files:

`src/drivers/uart_driver.h` — the header (API declarations)

`src/drivers/uart_driver.c` — the implementation

✓ VERIFY BEFORE CONTINUING

- ☐ Both files exist in src/drivers/

STEP 17 Create the watchdog driver

Open Document 3. Create both watchdog files:

`src/drivers/watchdog.h`

`src/drivers/watchdog.c`

✓ VERIFY BEFORE CONTINUING

- ☐ Both files exist in src/drivers/

STEP 18 Create CMakeLists.txt

Open Document 3. Find `CMakeLists.txt`. Create this file at the **root of your project**. This is your build system file.

✓ VERIFY BEFORE CONTINUING

- ☐ File exists at satellite-flight-computer/CMakeLists.txt (root level)

STEP 19 Create a minimal main.c to test the build

Before creating all the tasks, create a minimal `src/main.c` that just initialises UART and loops. This lets you confirm the build system works.

```
#include "uart_driver.h"
#include "FreeRTOS.h"
#include "task.h"

void vApplicationIdleHook(void) {}
void vApplicationStackOverflowHook(TaskHandle_t h, char *n) { (void)h; (void)n; for(;;){} }
void vApplicationMallocFailedHook(void) { for(;;){} }

int main(void) {
    uart_init(115200);
    uart_puts("\r\nBuild OK\r\n");
    vTaskStartScheduler();
    for(;;) {}
}
```

✓ VERIFY BEFORE CONTINUING

- ☐ src/main.c exists with the above content

STEP 20 Run the first CMake build

From the root of your project:

```
cmake -B build -G "Unix Makefiles"
cmake --build build
```

✓ VERIFY BEFORE CONTINUING

- ☐ cmake -B build completes with no errors (warnings are OK)
- ☐ cmake --build build produces build/satellite_fc.elf
- ☐ The size output shows text + data + bss sizes (all non-zero)

If you get "No rule to make target" errors: a file listed in CMakeLists.txt does not exist yet. Check that all files from Steps 12-19 were created at the exact paths listed.

STEP 21 Run the build in QEMU

```
qemu-system-arm -M lm3s6965evb -kernel build/satellite_fc.elf -serial mon:stdio -nographic
```

✓ VERIFY BEFORE CONTINUING

- ☐ Terminal prints: Build OK
- ☐ Press Ctrl+A then X to quit

PHASE 3 — Flight Computer Core**Weeks 3-4**
timeline**10-14 hrs**
hrs/phase

Replace the minimal `main.c` with the full flight computer implementation. Add all four tasks one at a time, verifying after each addition.

STEP 22 Create the ADCS task

Open Document 3. Find `src/tasks/adcs_task.c`. Create this file. This is the attitude determination task that runs at 50 Hz.

✓ VERIFY BEFORE CONTINUING

- ☐ File exists at `src/tasks/adcs_task.c`

STEP 23 Create the telemetry task

Open Document 3. Find `src/tasks/telemetry_task.c`. Create this file. This task reads ADCS state and transmits binary packets at 1 Hz.

✓ VERIFY BEFORE CONTINUING

- ☐ File exists at `src/tasks/telemetry_task.c`

STEP 24 Create the health monitor task

Open Document 3. Find `src/tasks/health_monitor.c`. Create this file. This is your highest-priority safety task.

✓ VERIFY BEFORE CONTINUING

- ☐ File exists at `src/tasks/health_monitor.c`

STEP 25 Create the command handler task

Open Document 3. Find `src/tasks/command_handler.c`. Create this file. This task receives single-byte uplink commands from the ground station.

✓ VERIFY BEFORE CONTINUING

- ☐ File exists at `src/tasks/command_handler.c`

STEP 26 Replace main.c with the full version

Open Document 3. Find `src/main.c`. **Delete your minimal `main.c` from Step 19** and replace it with the full version from Document 3. This wires all four tasks together.

✓ VERIFY BEFORE CONTINUING

- ☐ `src/main.c` now contains all 4 `xTaskCreate()` calls and the 3 hook functions

STEP 27 Rebuild and run the full flight computer

```
cmake --build build
qemu-system-arm -M lm3s6965evb -kernel build/satellite_fc.elf -serial mon:stdio -nographic
```

✓ VERIFY BEFORE CONTINUING

- ❑ Build succeeds with no errors
- ❑ Terminal output includes: "=== Satellite Flight Computer Booting ==="
- ❑ Terminal output includes: "Tasks created. Starting scheduler..."
- ❑ Terminal output includes: "[HM] tick=..." lines appearing every second
- ❑ Terminal output includes: "[CMD] Ready. Listening for uplink commands..."

You will also see binary garbage characters every second — that is the telemetry packet. This is expected. The binary data only makes sense when parsed by the Python ground station. Press Ctrl+A then X to quit QEMU.

STEP 28 Read and understand the ADCS timing

Open `src/tasks/adcs_task.c` and find the `vTaskDelayUntil` call. Read the comment above it. Understand why this gives true 50 Hz behaviour while `vTaskDelay` would not.

This is one of the most important concepts in the project. Be able to explain it in an interview.

`vTaskDelay(20ms)` = "sleep for 20ms from now" → task execution time causes drift
`vTaskDelayUntil(20ms)` = "wake at next 20ms boundary" → deterministic 50 Hz regardless of execution time

✓ VERIFY BEFORE CONTINUING

- ❑ You can explain the difference between `vTaskDelay` and `vTaskDelayUntil` without looking it up

PHASE 4 — Fault Detection & WatchdogWeek 5
timeline8-10 hrs
hrs/phase

The watchdog driver was already created in Phase 2. In this phase you confirm it is working correctly, verify the health monitor fault detection, and deliberately trigger a fault to see the recovery in action.

STEP 29 Verify watchdog is running

Run the flight computer in QEMU. Look at the output. The health monitor kicks the watchdog every 100ms. You should see regular `[HM] tick=...` lines, which confirms the watchdog is being fed.

✓ VERIFY BEFORE CONTINUING

- ☐ `[HM] tick=...` lines appear every second
- ☐ No `"[WDT] WATCHDOG EXPIRED"` message appears

STEP 30 Deliberately trigger a watchdog expiry (test)

To test that the watchdog actually works, temporarily comment out the `watchdog_kick()` call in `health_monitor.c`. Rebuild and run.

```
cmake --build build && qemu-system-arm -M lm3s6965evb -kernel build/satellite_fc.elf -
serial mon:stdio -nographic
```

✓ VERIFY BEFORE CONTINUING

- ☐ After 2 seconds: `"[WDT] WATCHDOG EXPIRED — system reset!"` appears
- ☐ System halts after that message
- ☐ IMPORTANT: Uncomment `watchdog_kick()` before continuing. Rebuild.

STEP 31 Deliberately trigger an ADCS fault (test)

To test the health monitor fault detection, temporarily add a long sleep inside the ADCS task loop to simulate it hanging. In `adcs_task.c`, inside the `for(;;)` loop, add: `vTaskDelay(pdMS_TO_TICKS(5000))` ;

Rebuild and run. The ADCS heartbeat will stop incrementing, and the health monitor should flag `FAULT_ADCS_TIMEOUT`.

✓ VERIFY BEFORE CONTINUING

- ☐ `"[HM] FAULT: ADCS heartbeat missed!"` appears in output
- ☐ The `[HM] faults=` field shows a non-zero value (0x01)
- ☐ IMPORTANT: Remove the `vTaskDelay` you added. Rebuild and confirm faults return to 0x00.

STEP 32 Review all fault flags and confirm they make sense

Open `src/common/flight_types.h`. Read every `FAULT_*` definition. Open `src/tasks/health_monitor.c` and trace through what triggers each fault and what clears it. Make sure you can explain each one.

✓ VERIFY BEFORE CONTINUING

- ☐ You understand what sets and clears each of the 5 fault flags

PHASE 5 — Python Ground Station GUI

Week 6
timeline8-10 hrs
hrs/phase

Build the Python ground station. This is the most visually impressive part of the project and the first thing you will show in an interview or a demo.

STEP 33 Create the telemetry parser

Open Document 3. Find `ground_station/telemetry_parser.py`. Create that file. This module converts raw binary bytes into Python dicts.

The TLM_FORMAT string "<HHIffffffBBhH" must exactly match TelemetryPacket_t in flight_types.h. Every field, in order, same types. If you change one, you must change the other.

✓ VERIFY BEFORE CONTINUING

- ☐ File exists at `ground_station/telemetry_parser.py`

STEP 34 Create an `__init__.py` so Python treats it as a package

Run this command from your project root:

```
touch ground_station/__init__.py
```

✓ VERIFY BEFORE CONTINUING

- ☐ `ground_station/__init__.py` exists (can be empty)

STEP 35 Write and run the parser unit tests

Open Document 3. Find `tests/test_parser.py`. Create that file.

Then run the tests:

```
pip3 install pytest
pytest tests/test_parser.py -v
```

✓ VERIFY BEFORE CONTINUING

- ☐ All 5 tests pass (green)
- ☐ No FAILED or ERROR lines

If you get `ImportError: No module named telemetry_parser` — check that `ground_station/__init__.py` exists and you ran `pytest` from the project root.

STEP 36 Create the ground station GUI

Open Document 3. Find `ground_station/gui.py`. Create that file. This is the full Tkinter + matplotlib dashboard.

✓ VERIFY BEFORE CONTINUING

- ☐ File exists at `ground_station/gui.py`

STEP 37 Test the GUI opens (without live data first)

Run the GUI with a port that does not exist yet, just to confirm it launches:

```
python3 ground_station/gui.py --port /tmp/gs_port
```

✓ VERIFY BEFORE CONTINUING

- ☐ A dark-themed window appears with telemetry labels and a plot area
- ☐ A CONNECT button is visible at the bottom
- ☐ The window does not crash immediately (it will show OFFLINE status — that is fine)
- ☐ Close the window when confirmed

STEP 38 Create the virtual serial port pair

You need two connected virtual serial ports: one for QEMU to write to, one for the GUI to read from. Run this in a **dedicated terminal window** and leave it running:

```
socat PTY,link=/tmp/fc_uart,rawer,echo=0 PTY,link=/tmp/gs_port,rawer,echo=0
```

✓ VERIFY BEFORE CONTINUING

- ☐ The socat command runs without error (it will appear to hang — that is correct, it is waiting)
- ☐ /tmp/fc_uart exists
- ☐ /tmp/gs_port exists

STEP 39 Run QEMU with the virtual serial port

In a **second terminal**:

```
qemu-system-arm -M lm3s6965evb -kernel build/satellite_fc.elf -serial /tmp/fc_uart -nographic
```

✓ VERIFY BEFORE CONTINUING

- ☐ QEMU runs without error
- ☐ Note: you will NOT see the usual text output in this terminal because serial output is going to /tmp/fc_uart, not to your screen. That is correct.

STEP 40 Connect the GUI to the live flight computer

In a **third terminal**:

```
python3 ground_station/gui.py --port /tmp/gs_port --baud 115200
```

Click the **CONNECT** button in the GUI.

✓ VERIFY BEFORE CONTINUING

- ☐ Status label changes to ONLINE (green)
- ☐ Roll, Pitch, Yaw values start updating every second
- ☐ The attitude plot shows a live line being drawn
- ☐ Fault field shows NOMINAL in green
- ☐ Packet count increments each second

STEP 41 Test command uplink

With everything running, click the **SAFE MODE** button in the GUI. Observe:

In the QEMU terminal: You will not see the mode change in QEMU output (the serial output is redirected). But in the GUI, the Mode field should change.

The `_send()` method in `gui.py` currently only prints to the Python console. To make it actually send bytes to the flight computer, you would extend `_send()` to do: `self.ser.write(cmd.encode())`. This is a known extension point for the bonus features phase.

 VERIFY BEFORE CONTINUING

- ☐ You understand how the command uplink path works end-to-end
- ☐ The GUI is visually impressive and functional

PHASE 6 — Polish, Testing & GitHub**Week 7**
timeline**6-8 hrs**
hrs/phase

This phase turns your working project into a portfolio-ready artifact. This is the phase that makes the difference between "I built this" and "I ship software like a professional".

STEP 42 Run the full test suite and confirm all pass

```
pytest tests/ -v --tb=short
```

✓ VERIFY BEFORE CONTINUING

- ☐ All tests pass
- ☐ Zero failures, zero errors

STEP 43 Run a full end-to-end smoke test

Run the complete system for at least 5 minutes and confirm it is stable:

Terminal 1: socat virtual port command (from Step 38)

Terminal 2: QEMU running the firmware

Terminal 3: Python GUI connected

✓ VERIFY BEFORE CONTINUING

- ☐ GUI runs for 5 minutes without crashing
- ☐ Packet count reaches at least 300 (5 min x 60 packets/min)
- ☐ Fault field stays NOMINAL throughout
- ☐ Roll/Pitch/Yaw values oscillate (not stuck at zero)
- ☐ No watchdog expiry message appears

STEP 44 Write the README.md

Create [README.md](#) at the project root. This is what a Boeing engineer will read when they visit your GitHub profile. Include all of these sections:

1. Project title and one-line description

RTOS-Based Satellite Flight Computer — FreeRTOS firmware simulating a real satellite attitude determination and control system, with UART telemetry and Python ground station.

2. Architecture diagram — a simple ASCII or image showing: 4 tasks, their rates, the queue between them, and the UART output path

3. Features list — each feature described in Boeing-aligned language:

- *Deterministic 50 Hz ADCS using vTaskDelayUntil (not vTaskDelay)*
- *Priority-inheritance mutexes preventing priority inversion*
- *Software watchdog timer with 2-second expiry and recovery handler*
- *Health monitor task with heartbeat checking and fault bitmask*
- *Binary telemetry over UART with magic number + XOR checksum validation*
- *Python ground station GUI with live matplotlib attitude plots*

4. Stack: FreeRTOS | C | ARM Cortex-M3 | QEMU | CMake | Python | Tkinter | pytest**5. How to build and run** — exact commands from git clone to running GUI**6. Concepts demonstrated** — write this out explicitly for keyword searchability:*Task scheduling | Priority inversion avoidance | ISR vs task separation |**Deterministic timing | Watchdog fault recovery | Binary protocol design | SIL testing***✓ VERIFY BEFORE CONTINUING**

- ☐ README.md exists at project root
- ☐ All 6 sections are present

STEP 45 Record a demo video or take screenshots

This is the single highest-impact thing you can do for your resume. A 30-second GIF or video of the live GUI with attitude plots updating is immediately striking to any reviewer.

Linux: Use [kazam](#) or [peek](#) to record a GIF. Install: `sudo apt install peek`

macOS: Use QuickTime (Cmd+Shift+5) to record the screen

Windows: Use Xbox Game Bar (Win+G) or OBS Studio (free)

Embed the GIF directly in your README.md:

```
![Ground Station Demo] (docs/demo.gif)
```

✓ VERIFY BEFORE CONTINUING

- ☐ A screenshot or GIF exists in docs/ folder
- ☐ It is referenced in README.md and renders when you view the README

STEP 46 Create a GitHub repository and push your project

```
# Create a .gitignore first
echo "build/\n*.o\n*.map\n__pycache__/\n*.pyc\n*.pytest_cache/" > .gitignore

# Stage and commit everything
git add .
git commit -m "Initial: RTOS satellite flight computer with FreeRTOS, QEMU, and Python GUI"

# Create a repo on github.com, then push:
git remote add origin https://github.com/YOUR_USERNAME/satellite-flight-computer.git
git push -u origin main
```

✓ VERIFY BEFORE CONTINUING

- ☐ Project is visible on github.com
- ☐ README renders correctly with all sections
- ☐ Demo image/GIF displays in the README
- ☐ build/ folder is NOT in the repository (confirmed by .gitignore)

STEP 47 Final resume line check

Copy this line exactly into your resume under Projects:

"Developed real-time embedded software using FreeRTOS on QEMU-simulated ARM Cortex-M3 hardware with deterministic 50 Hz task scheduling, priority-inheritance mutex design, software watchdog fault recovery, and binary UART telemetry — validated via a Python ground station GUI with live attitude plots and pytest-verified packet parsing."

Add the GitHub link directly below the resume line.

 VERIFY BEFORE CONTINUING

- ☐ Resume line is written using this exact phrasing (or your improved version)
- ☐ GitHub link is present and the repo is public
- ☐ You can speak confidently to every word in that resume line

Project Complete — Final Checklist

Run through this list before putting the project on your resume or showing it to anyone.

☐	BUILDS	cmake --build build completes with zero errors
☐	RUNS	QEMU boots, prints "Satellite Flight Computer Booting", shows [HM] status lines
☐	TELEMETRY	Binary packets transmit every second (visible as binary chars in terminal)
☐	GUI CONNECTS	Python GUI shows ONLINE, attitude values update, plot draws live
☐	FAULT TEST	Commenting out watchdog_kick() causes "[WDT] WATCHDOG EXPIRED" after 2s
☐	ADCS FAULT	Stalling ADCS task triggers FAULT_ADCS_TIMEOUT in health monitor output
☐	TESTS PASS	pytest tests/ -v shows all green
☐	README	README.md has all 6 sections, demo image, build instructions
☐	GITHUB	Repository is public, README renders, build/ is gitignored
☐	RESUME LINE	Resume line matches the phrasing in Step 47

47 steps. All done. Ship it.



RTOS Satellite Flight Computer | Document 4: Step-by-Step Process Guide